

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 2

По дисциплине «Обработка изображений в интеллектуальных системах»
Тема: «Конструирование моделей на базе предобученных нейронных сетей»

Выполнила:
Студентка 4 курса
Группы ИИ-25
Мархель А.Д.
Проверила:
Андренко К.В.

Брест 2026

Вариант 6

Цель: осуществлять обучение НС, сконструированных на базе предобученных архитектур НС.

Задачи лабораторной работы

1. Для заданной выборки и архитектуры предобученной нейронной сети организовать процесс обучения НС, предварительно изменив структуру слоев в соответствии с предложенной выборкой. Использовать тот же оптимизатор, что и в ЛР №1. Построить график изменения ошибки и оценить эффективность обучения на тестовой выборке;
2. Сравнить полученные результаты с результатами, полученными на кастомных архитектурах из ЛР №1;
3. Ознакомиться с state-of-the-art результатами для предлагаемых выборок (по материалам в сети Интернет). Сделать выводы о результатах обучения НС из п. 1 и 2;
4. Реализовать визуализацию работы предобученной СНС и кастомной (из ЛР 1). Визуализация осуществляется посредством выбора и подачи на сеть произвольного изображения с отображением результата классификации;
5. Оформить отчет по выполненной работе.

№ варианта	Выборка	Оптимизатор	Предобученная архитектура
6	MNIST	Adam	ResNet18

Ход работы

1. Адаптация и обучение предобученной сети ResNet18

ResNet18 — свёрточная нейронная сеть из 18 слоёв с весами (17 свёрточных и 1 полносвязный), предложенная He et al. в 2015 году. Сеть состоит из начального свёрточного слоя 7×7 , слоя подвыборки и четырёх групп остаточных блоков (64, 128, 256 и 512 каналов). В остаточных блоках вход блока складывается с его выходом (skip connection), что упрощает обучение глубоких сетей. После блоков выполняется глобальная усредняющая подвыборка и полносвязный слой классификации. В пакете torchvision

доступны веса ResNet18, предобученные на выборке ImageNet (1,2 млн цветных изображений 224×224, 1000 классов).

Для адаптации предобученной сети к выборке MNIST изменена структура сети и выполнено переформатирование данных:

- заменён выходной полносвязный слой: вместо Linear(512→1000) для 1000 классов ImageNet установлен новый слой Linear(512→10) по числу классов MNIST (цифры 0–9);
- изображения MNIST увеличены с 28×28 до 224×224 пикселей (transforms.Resize) — размер, на котором сеть обучалась на ImageNet;
- одноканальные изображения преобразованы в трёхканальные (transforms.Grayscale с num_output_channels=3): яркость копируется в каналы R, G и B. Это позволяет сохранить предобученные веса первого свёрточного слоя, рассчитанного на цветные изображения;
- выполнена нормализация средним (0,485; 0,456; 0,406) и СКО (0,229; 0,224; 0,225) выборки ImageNet, как рекомендуется в документации torchvision для предобученных моделей.

Параметры обучения: функция потерь CrossEntropyLoss оптимизатор Adam (как в ЛР №1) с шагом обучения 0.0001 размер мини-батча 64 число эпох 5. Дообучаются все слои сети число обучаемых параметров — 11 181 642. Обучение выполнялось на видеокарте NVIDIA GeForce RTX 3050 Ti с использованием смешанной точности вычислений (torch.autocast).

Код программы

models.py

```
import torch.nn as nn
from torchvision import transforms
from torchvision.models import ResNet18_Weights, resnet18

IMAGE_SIZE = 224
IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD = [0.229, 0.224, 0.225]
MNIST_MEAN, MNIST_STD = 0.1307, 0.3081

resnet_transform = transforms.Compose([
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
    transforms.Grayscale(num_output_channels=3),
    transforms.ToTensor(),
    transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
])

custom_transform = transforms.Compose([
    transforms.ToTensor(),
```

```

        transforms.Normalize((MNIST_MEAN,), (MNIST_STD,)),
    ])

def build_resnet18(num_classes=10, pretrained=True):
    weights = ResNet18_Weights.IMAGENET1K_V1 if pretrained else None
    model = resnet18(weights=weights)
    model.fc = nn.Linear(model.fc.in_features, num_classes)
    return model

class SimpleCNN(nn.Module):

    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 7 * 7, 128),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(128, num_classes),
        )

    def forward(self, x):
        return self.classifier(self.features(x))

```

main.py

```

"""
Лабораторная работа №2. Конструирование моделей на базе предобученных нейронных сетей.
Вариант 6: выборка MNIST, оптимизатор Adam, предобученная архитектура ResNet18.
"""

import json
import os
import random
import time

import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import datasets

```

```
from models import (IMAGENET_MEAN, IMAGENET_STD, SimpleCNN, build_resnet18,
                    custom_transform, resnet_transform)
```

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(BASE_DIR, "data")
RESULTS_DIR = os.path.join(BASE_DIR, "results")
MODEL_PATH = os.path.join(BASE_DIR, "mnist_resnet18.pth")
LAB1_MODEL_PATH = os.path.join(BASE_DIR, "lab1", "mnist_cnn.pth")
LAB1_HISTORY_PATH = os.path.join(BASE_DIR, "lab1", "history.json")
```

```
EPOCHS = 5
BATCH_SIZE = 64
LEARNING_RATE = 1e-4
NUM_WORKERS = 2
SEED = 42
```

```
def set_seed(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
```

```
def get_loaders(transform, batch_size, workers):
    train_set = datasets.MNIST(DATA_DIR, train=True, download=True, transform=transform)
    test_set = datasets.MNIST(DATA_DIR, train=False, download=True, transform=transform)
    train_loader = DataLoader(train_set, batch_size=batch_size, shuffle=True,
                              num_workers=workers,
                              pin_memory=True, persistent_workers=workers > 0)
    test_loader = DataLoader(test_set, batch_size=256, shuffle=False, num_workers=workers,
                             pin_memory=True, persistent_workers=workers > 0)
    return train_set, test_set, train_loader, test_loader
```

```
def train_one_epoch(model, loader, criterion, optimizer, scaler, device):
    model.train()
    total_loss, correct, total = 0.0, 0, 0
    for images, labels in loader:
        images, labels = images.to(device, non_blocking=True), labels.to(device,
non_blocking=True)
        optimizer.zero_grad()
        with torch.autocast(device_type=device.type, enabled=device.type == "cuda"):
            outputs = model(images)
            loss = criterion(outputs, labels)
        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()

        total_loss += loss.item() * images.size(0)
        correct += (outputs.argmax(1) == labels).sum().item()
        total += images.size(0)
```

```

    return total_loss / total, 100.0 * correct / total

@torch.no_grad()
def evaluate(model, loader, criterion, device):
    model.eval()
    total_loss, correct, total = 0.0, 0, 0
    for images, labels in loader:
        images, labels = images.to(device, non_blocking=True), labels.to(device,
non_blocking=True)
        with torch.autocast(device_type=device.type, enabled=device.type == "cuda"):
            outputs = model(images)
            loss = criterion(outputs, labels)
            total_loss += loss.item() * images.size(0)
            correct += (outputs.argmax(1) == labels).sum().item()
            total += images.size(0)
    return total_loss / total, 100.0 * correct / total

def plot_history(history, path):
    epochs = range(1, len(history["train_loss"]) + 1)
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4.5))

    ax1.plot(epochs, history["train_loss"], "o-", label="Train Loss")
    ax1.plot(epochs, history["test_loss"], "s-", label="Test Loss")
    ax1.set_title("Ошибка (CrossEntropyLoss)")
    ax1.set_xlabel("Эпоха")
    ax1.set_ylabel("Loss")
    ax1.set_xticks(list(epochs))
    ax1.grid(alpha=0.3)
    ax1.legend()

    ax2.plot(epochs, history["train_acc"], "o-", label="Train Accuracy")
    ax2.plot(epochs, history["test_acc"], "s-", label="Test Accuracy")
    ax2.set_title("Точность, %")
    ax2.set_xlabel("Эпоха")
    ax2.set_ylabel("Accuracy, %")
    ax2.set_xticks(list(epochs))
    ax2.grid(alpha=0.3)
    ax2.legend()

    fig.suptitle("Обучение ResNet18 на MNIST (оптимизатор Adam)")
    fig.tight_layout()
    fig.savefig(path, dpi=150)
    return fig

def plot_comparison(history, lab1_history, path):
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4.5))
    for hist, name, marker in [(lab1_history, "Кастомная ЧС (ЛР1)", "o-"), (history, "ResNet18",
"s-")]:

```

```

        epochs = range(1, len(hist["test_loss"]) + 1)
        ax1.plot(epochs, hist["test_loss"], marker, label=name)
        ax2.plot(epochs, hist["test_acc"], marker, label=name)
    ax1.set_title("Ошибка на тестовой выборке")
    ax1.set_xlabel("Эпоха")
    ax1.set_ylabel("Loss")
    ax2.set_title("Точность на тестовой выборке, %")
    ax2.set_xlabel("Эпоха")
    ax2.set_ylabel("Accuracy, %")
    for ax in (ax1, ax2):
        ax.set_xticks(range(1, max(len(history["test_loss"]), len(lab1_history["test_loss"]))) +
1))
        ax.grid(alpha=0.3)
        ax.legend()
    fig.suptitle("Сравнение ResNet18 и кастомной СНС из ЛР1")
    fig.tight_layout()
    fig.savefig(path, dpi=150)
    return fig

```

```

@torch.no_grad()
def plot_test_predictions(model, test_set, device, path, count=12):
    model.eval()
    indices = random.sample(range(len(test_set)), count)
    mean = torch.tensor(IMAGENET_MEAN).view(3, 1, 1)
    std = torch.tensor(IMAGENET_STD).view(3, 1, 1)
    fig, axes = plt.subplots(2, count // 2, figsize=(14, 6))
    for ax, idx in zip(axes.flat, indices):
        image, label = test_set[idx]
        probs = torch.softmax(model(image.unsqueeze(0).to(device)).float(), dim=1)[0]
        pred = probs.argmax().item()
        ax.imshow((image * std + mean).clamp(0, 1).permute(1, 2, 0))
        color = "green" if pred == label else "red"
        ax.set_title(f"True: {label}\nPred: {pred} ({probs[pred]:.1%})", color=color,
    fontsize=10)
        ax.axis("off")
    fig.suptitle("Классификация изображений тестовой выборки сетью ResNet18 (вход 3x224x224)")
    fig.tight_layout()
    fig.savefig(path, dpi=150)
    return fig

```

```

def evaluate_lab1_model(criterion, device):
    model = SimpleCNN().to(device)
    model.load_state_dict(torch.load(LAB1_MODEL_PATH, map_location=device, weights_only=True))
    _, _, _, test_loader = get_loaders(custom_transform, BATCH_SIZE, 0)
    return evaluate(model, test_loader, criterion, device)

```

```

def main():
    set_seed(SEED)
    os.makedirs(RESULTS_DIR, exist_ok=True)

```

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Устройство: {torch.cuda.get_device_name(0) if device.type == 'cuda' else 'cpu'}")

train_set, test_set, train_loader, test_loader = get_loaders(resnet_transform, BATCH_SIZE,
NUM_WORKERS)
print(f"Обучающая выборка: {len(train_set)} изображений, тестовая: {len(test_set)}")
print(f"Размер входа сети: {tuple(train_set[0][0].shape)}")

model = build_resnet18(num_classes=10, pretrained=True).to(device)
print(f"Новый выходной слой: {model.fc}")
n_params = sum(p.numel() for p in model.parameters())
print(f"Число обучаемых параметров: {n_params}")

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
scaler = torch.amp.GradScaler(device.type, enabled=device.type == "cuda")

history = {"train_loss": [], "train_acc": [], "test_loss": [], "test_acc": [], "epoch_time":
[]}]
log_lines = []
for epoch in range(1, EPOCHS + 1):
    start = time.time()
    train_loss, train_acc = train_one_epoch(model, train_loader, criterion, optimizer,
scaler, device)
    test_loss, test_acc = evaluate(model, test_loader, criterion, device)
    elapsed = time.time() - start
    for key, value in zip(history, (train_loss, train_acc, test_loss, test_acc, elapsed)):
        history[key].append(value)
    line = (f"Epoch {epoch}/{EPOCHS} | Train Loss: {train_loss:.4f} | Train Acc:
{train_acc:.2f}% "
f"| Test Loss: {test_loss:.4f} | Test Acc: {test_acc:.2f}% | Time: {elapsed:.0f}
s")
    print(line, flush=True)
    log_lines.append(line)

best_epoch = int(np.argmax(history["test_acc"])) + 1
summary = (f"ResNet18: итоговая точность на тестовой выборке {history['test_acc'][-1]:.2f}% "
f"(ошибка классификации {100 - history['test_acc'][-1]:.2f}%). "
f"Лучшая: {max(history['test_acc']):.2f}% на эпохе {best_epoch}")
print(summary)
log_lines.append(summary)

torch.save(model.state_dict(), MODEL_PATH)
print(f"Модель сохранена: {MODEL_PATH}")

lab1_loss, lab1_acc = evaluate_lab1_model(criterion, device)
comparison = (f"Кастомная СНС (ЛР1): точность на тестовой выборке {lab1_acc:.2f}%, "
f"ошибка {lab1_loss:.4f}")
print(comparison)
log_lines.append(comparison)

```



```

with open(os.path.join(RESULTS_DIR, "training_log.txt"), "w", encoding="utf-8") as f:
    f.write("\n".join(log_lines) + "\n")
with open(os.path.join(RESULTS_DIR, "history.json"), "w", encoding="utf-8") as f:
    json.dump({"history": history, "params": n_params, "epochs": EPOCHS, "batch_size":
BATCH_SIZE,
              "lr": LEARNING_RATE, "device": str(device), "lab1_test_acc": lab1_acc,
              "lab1_test_loss": lab1_loss}, f, indent=2)

plot_history(history, os.path.join(RESULTS_DIR, "loss_accuracy.png"))
with open(LAB1_HISTORY_PATH, encoding="utf-8") as f:
    plot_comparison(history, json.load(f)["history"], os.path.join(RESULTS_DIR,
"comparison.png"))
    plot_test_predictions(model, test_set, device, os.path.join(RESULTS_DIR,
"test_predictions.png"))
    plt.show()

if __name__ == "__main__":
    main()

```

predict.py

"""

Визуализация работы предобученной ResNet18 и кастомной СНС из ЛР1.

Запуск:

python predict.py - откроется окно выбора файла изображения

python predict.py путь/к/файлу - классифицировать указанный файл

Если файл не выбран, берётся случайное изображение из тестовой выборки MNIST.

"""

```

import os
import random
import sys

```

```

import matplotlib.pyplot as plt
import numpy as np
import torch
from PIL import Image, ImageOps
from torchvision import datasets

```

```

from models import SimpleCNN, build_resnet18, custom_transform, resnet_transform

```

```

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(BASE_DIR, "data")
RESULTS_DIR = os.path.join(BASE_DIR, "results")
RESNET_PATH = os.path.join(BASE_DIR, "mnist_resnet18.pth")
CUSTOM_PATH = os.path.join(BASE_DIR, "lab1", "mnist_cnn.pth")

```

```

def choose_file():
    try:

```

```

        import tkinter as tk
        from tkinter import filedialog
    except ImportError:
        return None
    root = tk.Tk()
    root.withdraw()
    root.attributes("-topmost", True)
    path = filedialog.askopenfilename(
        title="Выберите изображение цифры",
        filetypes=[("Изображения", "*.png *.jpg *.jpeg *.bmp *.gif *.webp"), ("Все файлы",
        "**.*")],
    )
    root.destroy()
    return path or None

```

```

def to_mnist_format(image):
    img = image.convert("L")
    arr = np.asarray(img, dtype=np.float32)
    if np.median(arr) > 127:
        arr = 255 - arr

    background = np.median(arr)
    arr = np.clip((arr - background) / max(arr.max() - background, 1.0), 0, 1)
    arr = np.clip((arr - 0.3) / 0.7, 0, 1) * 255
    img = Image.fromarray(arr.astype(np.uint8))

    mask = arr > 50
    if mask.any():
        rows, cols = np.where(mask)
        img = img.crop((cols.min(), rows.min(), cols.max() + 1, rows.max() + 1))

    w, h = img.size
    scale = 20.0 / max(w, h)
    img = img.resize((max(1, round(w * scale)), max(1, round(h * scale))), Image.LANCZOS)
    canvas = Image.new("L", (28, 28), 0)
    canvas.paste(img, ((28 - img.width) // 2, (28 - img.height) // 2))
    return canvas

```

```

def load_models():
    for path in (RESNET_PATH, CUSTOM_PATH):
        if not os.path.exists(path):
            sys.exit(f"Файл модели не найден: {path}. Сначала запустите main.py.")
    resnet = build_resnet18(num_classes=10, pretrained=False)
    resnet.load_state_dict(torch.load(RESNET_PATH, map_location="cpu", weights_only=True))
    custom = SimpleCNN()
    custom.load_state_dict(torch.load(CUSTOM_PATH, map_location="cpu", weights_only=True))
    return resnet.eval(), custom.eval()

```

```

@torch.no_grad()
def classify(model, tensor):
    return torch.softmax(model(tensor.unsqueeze(0)), dim=1)[0].numpy()

def plot_probs(ax, probs, title):
    pred = int(probs.argmax())
    ax.bar(range(10), probs, color=["tab:green" if d == pred else "tab:blue" for d in range(10)])
    ax.set_xticks(range(10))
    ax.set_ylim(0, 1)
    ax.set_xlabel("Цифра")
    ax.set_ylabel("Вероятность")
    ax.set_title(f"{title}\nРезультат: {pred} ({probs[pred]:.1%})")

def show_result(resnet, custom, original, title, save_path=None):
    prepared = to_mnist_format(original)
    resnet_probs = classify(resnet, resnet_transform(prepared))
    custom_probs = classify(custom, custom_transform(prepared))

    print(title)
    for name, probs in (("ResNet18", resnet_probs), ("Кастомная СНС (ЛР1)", custom_probs)):
        pred = int(probs.argmax())
        print(f" {name}: цифра {pred} (уверенность {probs[pred]:.2%})")

    fig, axes = plt.subplots(1, 4, figsize=(16, 4.5), gridspec_kw={"width_ratios": [1, 1, 1.4,
1.4]})
    axes[0].imshow(np.asarray(original.convert("L")), cmap="gray")
    axes[0].set_title("Исходное изображение")
    axes[1].imshow(np.asarray(prepared), cmap="gray")
    axes[1].set_title("После предобработки 28x28")
    for ax in axes[2:]:
        ax.axis("off")
    plot_probs(axes[2], resnet_probs, "ResNet18 (вход 3x224x224)")
    plot_probs(axes[3], custom_probs, "Кастомная СНС из ЛР1 (вход 1x28x28)")
    fig.suptitle(title)
    fig.tight_layout()
    if save_path:
        fig.savefig(save_path, dpi=150)
    plt.show()

def main():
    resnet, custom = load_models()
    path = sys.argv[1] if len(sys.argv) > 1 else choose_file()

    if path:
        original = Image.open(path)
        title = f"Файл: {os.path.basename(path)}"
        name = os.path.splitext(os.path.basename(path))[0]
    else:

```

```

test_set = datasets.MNIST(DATA_DIR, train=False, download=True)
idx = random.randrange(len(test_set))
original, label = test_set[idx]
title = f"Тестовое изображение MNIST №{idx}, истинная цифра: {label}"
name = f"mnist_{idx}"

os.makedirs(RESULTS_DIR, exist_ok=True)
show_result(resnet, custom, original, title, os.path.join(RESULTS_DIR,
f"prediction_{name}.png"))

if __name__ == "__main__":
    main()

```

Результат:

Вывод программы в консоли при обучении:

```

Epoch 1/5 | Train Loss: 0.0647 | Train Acc: 98.23% | Test Loss: 0.0216 | Test Acc: 99.34% | Time: 207 s
Epoch 2/5 | Train Loss: 0.0190 | Train Acc: 99.43% | Test Loss: 0.0194 | Test Acc: 99.43% | Time: 181 s
Epoch 3/5 | Train Loss: 0.0135 | Train Acc: 99.59% | Test Loss: 0.0171 | Test Acc: 99.41% | Time: 191 s
Epoch 4/5 | Train Loss: 0.0113 | Train Acc: 99.64% | Test Loss: 0.0295 | Test Acc: 99.24% | Time: 193 s
Epoch 5/5 | Train Loss: 0.0085 | Train Acc: 99.73% | Test Loss: 0.0181 | Test Acc: 99.45% | Time: 191 s
ResNet18: итоговая точность на тестовой выборке 99.45% (ошибка классификации 0.55%). Лучшая: 99.45% на эпохе 5
Кастомная СНС (ЛР1): точность на тестовой выборке 99.25%, ошибка 0.0266

```

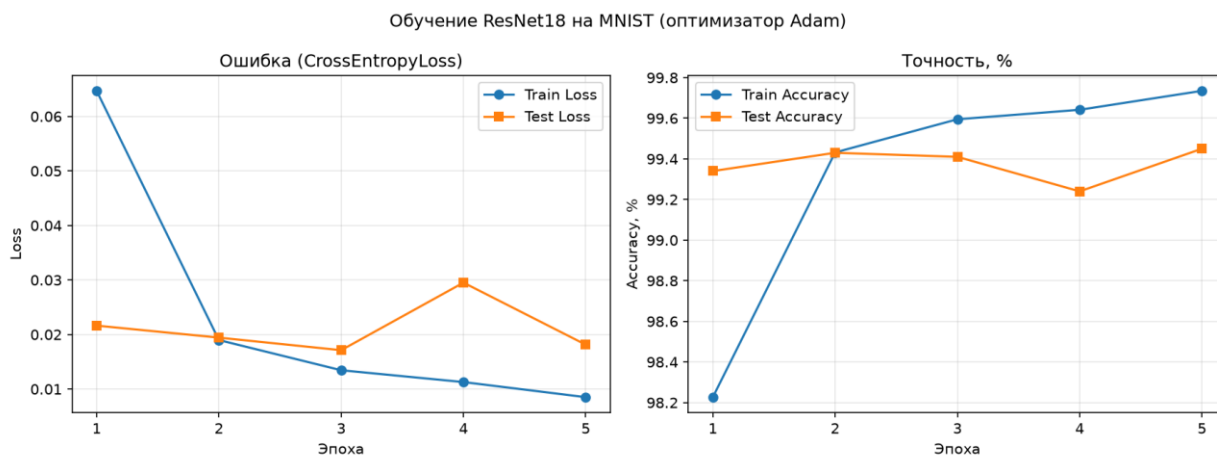


Рисунок 1 – Графики изменения ошибки и точности ResNet18 на обучающей и тестовой выборках

После 5 эпох точность ResNet18 на тестовой выборке составила 99,45% (ошибка классификации 0,55%, т.е. 55 неверно распознанных изображений из

10 000), значение функции потерь на тесте — 0,0181. Наилучшая точность 99,45% достигнута на эпохе 5. Уже после первой эпохи точность на тесте составила 99,34%.

Ошибка на обучающей выборке монотонно снижается на протяжении всех эпох (с 0,0647 до 0,0085), а точность на обучающей выборке растёт до 99,73%. Ошибка на тестовой выборке минимальна на 3-й эпохе (0,0171); на 4-й эпохе наблюдается кратковременный скачок ошибки (0,0295) с падением точности до 99,24%, после чего на 5-й эпохе сеть вернулась к точности 99,45%. Такие колебания характерны для дообучения большой сети с адаптивным оптимизатором при постоянном шаге обучения. Начиная со 2-й эпохи точность на обучающей выборке превышает тестовую — это признак начинающегося переобучения, поэтому увеличение числа эпох без снижения шага обучения или аугментации данных не даст существенного прироста точности.

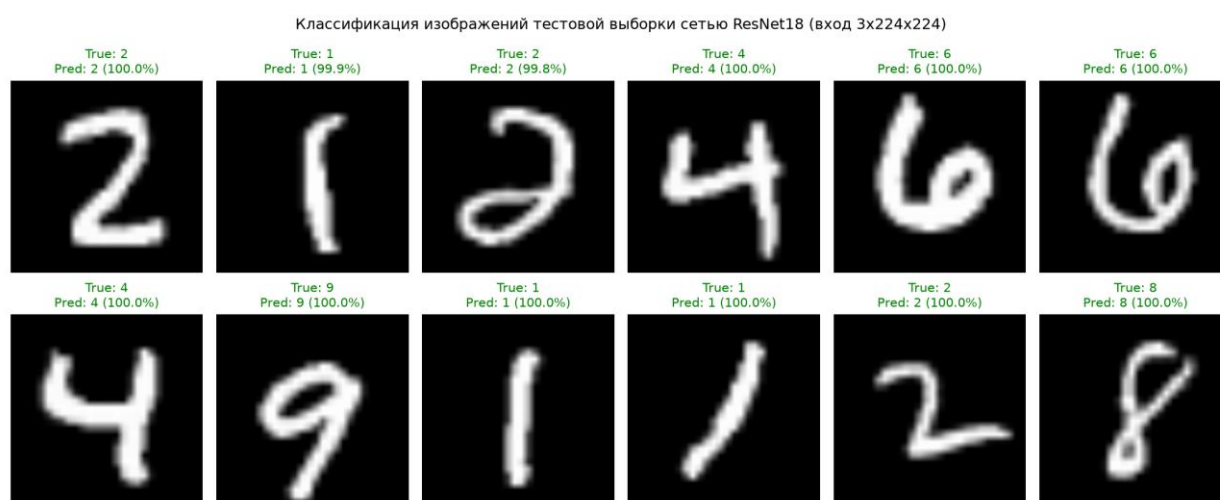


Рисунок 2 – Классификация изображений тестовой выборки сетью ResNet18 (изображения после предобработки: 3×224×224)

2. Сравнение с кастомной архитектурой из ЛР №1

Показатель	Кастомная СНС (ЛР №1)	ResNet18 (ЛР №2)
Архитектура	2 свёрточных + 2 полносвязных слоя	18 слоёв, остаточные блоки
Начальные веса	случайные	предобученные на ImageNet
Размер входа	1×28×28	3×224×224
Число параметров	421 642	11 181 642
Оптимизатор	Adam, lr = 0,001	Adam, lr = 0,0001

Число эпох	10	5
Устройство обучения	CPU	GPU RTX 3050 Ti
Точность после 1-й эпохи, %	98,57	99,34
Итоговая точность на тесте, %	99,25	99,45
Ошибка классификации, %	0,75	0,55
Функция потерь на тесте	0,0266	0,0181

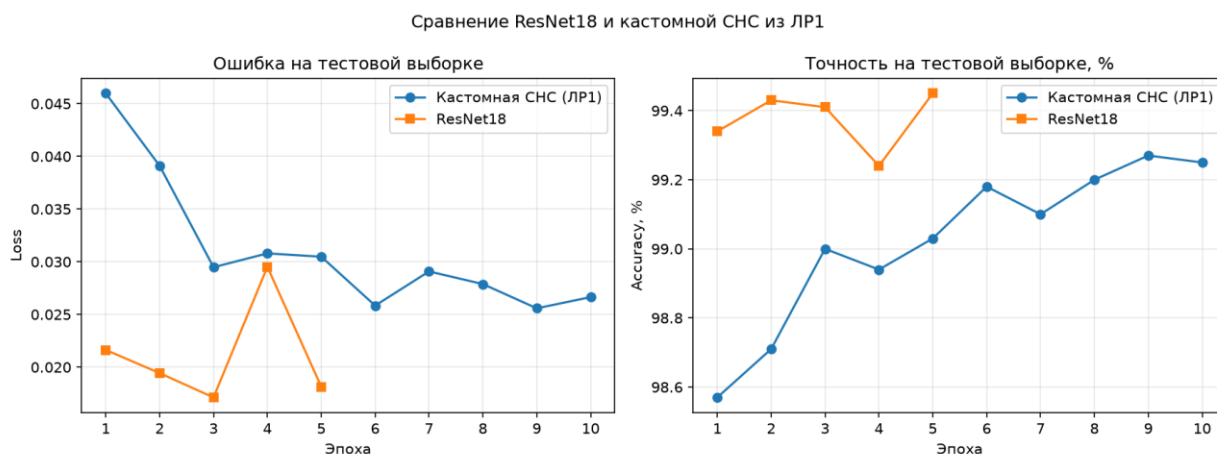


Рисунок 3 – Сравнение ошибки и точности на тестовой выборке ResNet18 и кастомной СНС из ЛР №1

Дообученная ResNet18 превзошла кастомную СНС из ЛР №1 по всем показателям качества: точность на тестовой выборке выросла с 99,25% до 99,45%, число ошибок сократилось с 75 до 55 изображений из 10 000 (примерно на 27%), значение функции потерь на тесте снизилось с 0,0266 до 0,0181. Особенно заметно преимущество предобученной сети в начале обучения: уже после первой эпохи ResNet18 показала 99,34%, тогда как кастомной сети для достижения 99,25% потребовалось 10 эпох.

Высокая скорость сходимости объясняется переносом знаний (transfer learning): свёрточные слои ResNet18, обученные на ImageNet, уже умеют выделять универсальные признаки — края, штрихи, изгибы и их сочетания, поэтому сети требуется лишь настроить их под цифры MNIST и обучить новый выходной слой.

Цена этого выигрыша — значительно большие вычислительные затраты: ResNet18 содержит в 26,5 раза больше параметров (11,18 млн против 0,42 млн), обрабатывает изображения $3 \times 224 \times 224$ вместо $1 \times 28 \times 28$ (в 192 раза больше входных значений) и требует видеокарты — одна эпоха на GPU занимала около

3 минут. Кастомная СНС при этом уступает лишь 0,2 процентного пункта точности. Следовательно, для простых выборок вроде MNIST небольшая специализированная сеть является более экономичным решением, а предобученные архитектуры наиболее выгодны на сложных выборках и при малом объеме обучающих данных.

3. Сравнение с state-of-the-art результатами

Модель	Точность, %	Ошибка, %
Ансамбль СНС с гомогенными векторными капсулами (Byerly et al., 2021)	99,87	0,13
Ансамбль СНС с DropConnect (Wan et al., 2013)	99,79	0,21
Многоколоночная глубокая СНС MCDNN (Cireşan et al., 2012)	99,77	0,23
LeNet-5 (LeCun et al., 1998)	99,05	0,95
ResNet18, дообученная (данная работа)	99,45	0,55
Кастомная СНС (ЛР №1)	99,25	0,75

Точность дообученной ResNet18 (99,45%) на 0,42 процентного пункта ниже лучшего известного результата на MNIST (99,87%) и выше классической сети LeNet-5. Отставание от state-of-the-art объясняется тем, что лучшие результаты получены ансамблями из нескольких сетей, с аугментацией данных (сдвиги, повороты, эластичные искажения) и длительным обучением с уменьшением шага, тогда как в данной работе использовалась одна сеть, 5 эпох и постоянный шаг обучения без аугментации.

На выборке MNIST все современные методы находятся в узком диапазоне 99,0–99,9%, поскольку выборка относительно проста, а оставшиеся ошибки приходится на неоднозначно написанные цифры. Поэтому разница между кастомной сетью и ResNet18 (0,2 п.п.) невелика в абсолютном выражении, но соответствует заметному сокращению числа ошибок.

4. Визуализация работы предобученной и кастомной СНС

Для визуализации реализован скрипт `predict.py`. При запуске открывается окно выбора файла, в котором можно выбрать произвольное изображение цифры (png, jpg, bmp и др.), например найденное в сети Интернет или сфотографированное. Изображение переводится в оттенки серого, при светлом фоне инвертируется (в MNIST цифры белые на чёрном фоне), фон выравнивается, цифра обрезается по границам, вписывается в квадрат 20×20 и центрируется на поле 28×28 — так же, как подготовлены изображения MNIST.

Далее одно и то же изображение подаётся на обе сети: на кастомную СНС из ЛР №1 — в исходном формате $1 \times 28 \times 28$, на ResNet18 — после той же предобработки, что и при обучении (увеличение до 224×224 , три канала, нормализация ImageNet). На экран выводятся исходное изображение, изображение после предобработки и вероятности классов для каждой сети с итоговым результатом классификации. Если файл не выбран, берётся случайное изображение тестовой выборки MNIST.

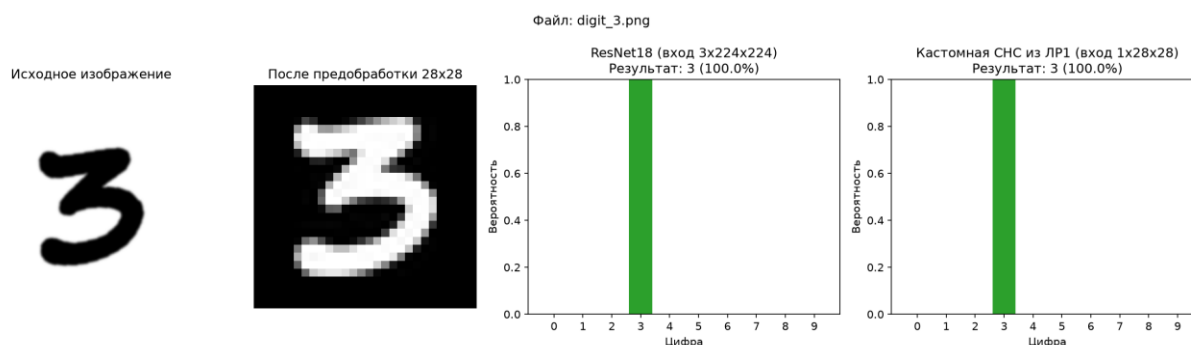


Рисунок 4 – Результат распознавания изображения digit_3

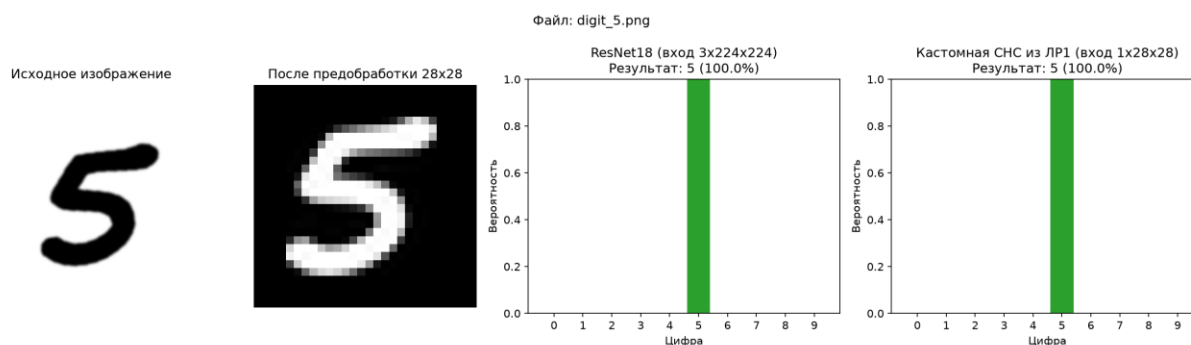


Рисунок 5 – Результат распознавания изображения digit_5

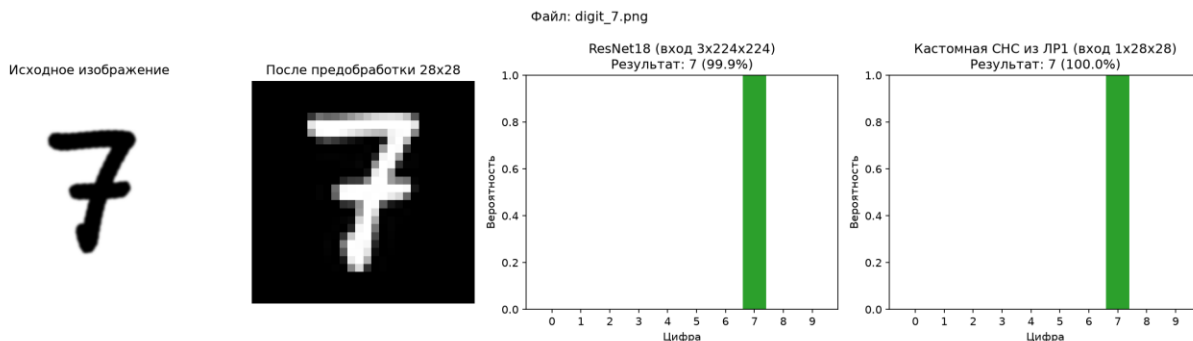


Рисунок 6 – Результат распознавания изображения digit_7

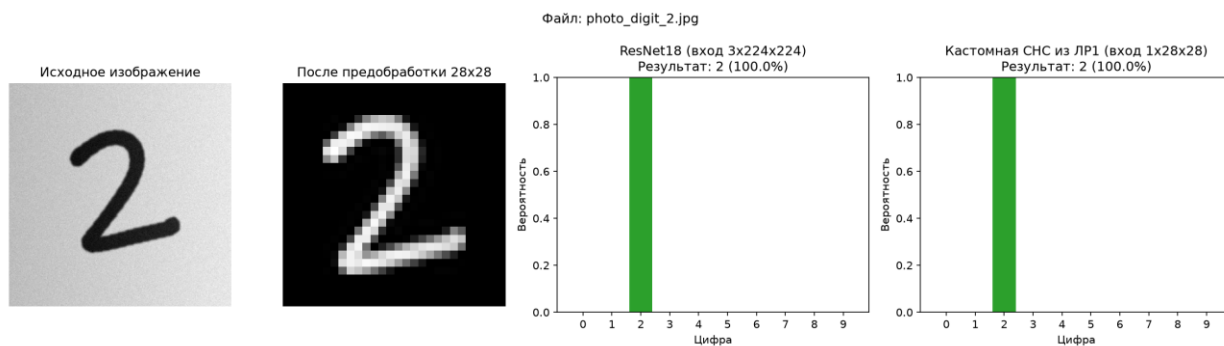


Рисунок 7 – Результат распознавания изображения photo_digit_2

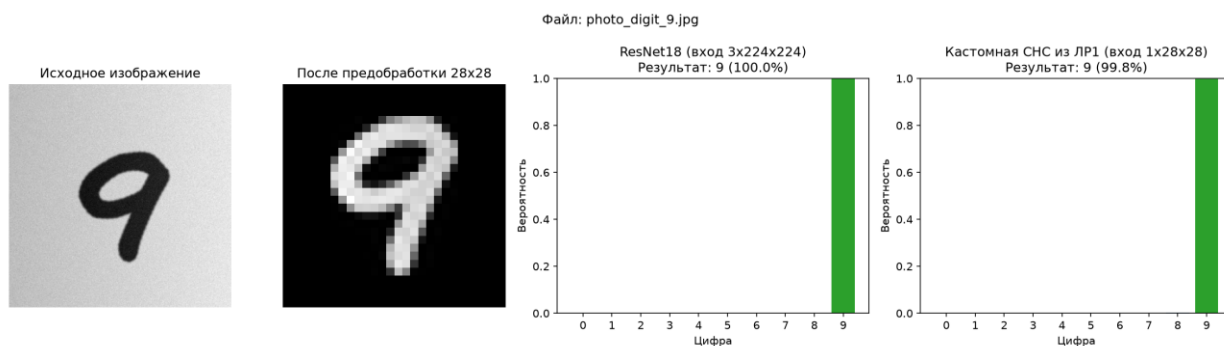


Рисунок 8 – Результат распознавания изображения photo_digit_9

Вывод программы predict.py в консоли:

Файл: digit_3.png

ResNet18: цифра 3 (уверенность 100.00%)

Кастомная СНС (ЛР1): цифра 3 (уверенность 100.00%)

Файл: digit_5.png

ResNet18: цифра 5 (уверенность 100.00%)

Кастомная СНС (ЛР1): цифра 5 (уверенность 100.00%)

Файл: digit_7.png

ResNet18: цифра 7 (уверенность 99.95%)

Кастомная СНС (ЛР1): цифра 7 (уверенность 99.98%)

Файл: photo_digit_2.jpg

ResNet18: цифра 2 (уверенность 100.00%)
Кастомная СНС (ЛР1): цифра 2 (уверенность 100.00%)

Файл: photo_digit_9.jpg

ResNet18: цифра 9 (уверенность 99.99%)
Кастомная СНС (ЛР1): цифра 9 (уверенность 99.83%)

Для проверки были подготовлены изображения цифр, написанные рукописными шрифтами, в том числе изображения, имитирующие фотографию цифры на бумаге (серый неравномерный фон, шум, цветовой оттенок). Обе сети верно распознали все изображения. ResNet18 классифицировала их с уверенностью не ниже 99,95%, кастомная СНС — не ниже 99,83%. Корректная предобработка (инверсия, удаление фона, центрирование) позволяет применять обе сети к изображениям, которые по стилю отличаются от выборки MNIST.

Использование предобученной архитектуры ResNet18 позволило с минимальными изменениями структуры сети — заменой выходного слоя на 10 классов — и переформатированием данных (увеличение до 224×224, дублирование канала яркости в три канала, нормализация ImageNet) получить классификатор рукописных цифр с точностью 99,45%. Предобученные признаки ImageNet хорошо переносятся даже на одноканальные изображения цифр, благодаря чему сеть достигла высокой точности уже на первой эпохе.

По сравнению с кастомной СНС из ЛР №1 предобученная сеть точнее и быстрее сходится по числу эпох, но требует в десятки раз больше параметров и вычислений. Для дальнейшего повышения точности целесообразно применить аугментацию данных, уменьшение шага обучения по расписанию и ансамблирование моделей.

Вывод: проделав данную лабораторную работу, я научилась осуществлять обучение нейронных сетей, сконструированных на базе предобученных архитектур. Предобученная сеть ResNet18, адаптированная к выборке MNIST и дообученная с оптимизатором Adam, достигла точности 99,45% на тестовой выборке, что выше точности кастомной СНС из ЛР №1 (99,25%).